

~~AIPEN~~—AI-Assisted Web Application Penetration-Testing Platform

Project Requirements, Design & SDLC Plan

Field	Value
Document version	1.0
Date	2026-09-16
Author	Project owner (solo developer)
Status	Approved for Phase 1
Formats	Markdown (source of truth), PDF (rendered copy)

Table of Contents

1. [Introduction](#1-introduction)
2. [Project Overview](#2-project-overview)
3. [Planning Phase](#3-planning-phase)
4. [Requirements Specification](#4-requirements-specification)
5. [System Design](#5-system-design)
6. [Implementation Plan](#6-implementation-plan)
7. [Testing Strategy](#7-testing-strategy)
8. [Evaluation Methodology](#8-evaluation-methodology)
9. [Ethics, Legal & Safety](#9-ethics-legal--safety)
10. [Risk Management](#10-risk-management)
11. [Future Work & Commercialization](#11-future-work--commercialization)
12. [Appendices](#12-appendices)

1. Introduction

1.1 Background

Web application penetration testing is a high-skill, time-consuming activity. Existing automated scanners (Nuclei, OWASP ZAP, sqlmap) are excellent at generating *signals* — raw findings, response anomalies, endpoint inventories — but they suffer from three structural weaknesses:

1. **High false-positive rates.** Scanners report pattern matches, not verified vulnerabilities. A human must manually triage the majority of scanner output.
2. **No reasoning over evidence.** A scanner cannot decide that an HTTP 500 caused by a single-quote probe *plus* a UNION-based extraction probe *plus* a time-based delay constitutes confirmed SQL injection. Only a reasoning agent can chain evidence into a conclusion.
3. **No attack planning.** Scanners run their templates indiscriminately. They do not prioritise "the upload endpoint is unauthenticated and accepts PDF — test file-type validation first" based on an understanding of the application.

At the same time, naive "AI pentesting" products that pipe a target directly into an LLM and ask it to "find vulnerabilities" are little more than wrappers: they hallucinate findings, cannot distinguish evidence from speculation, and are dangerous in uncontrolled hands.

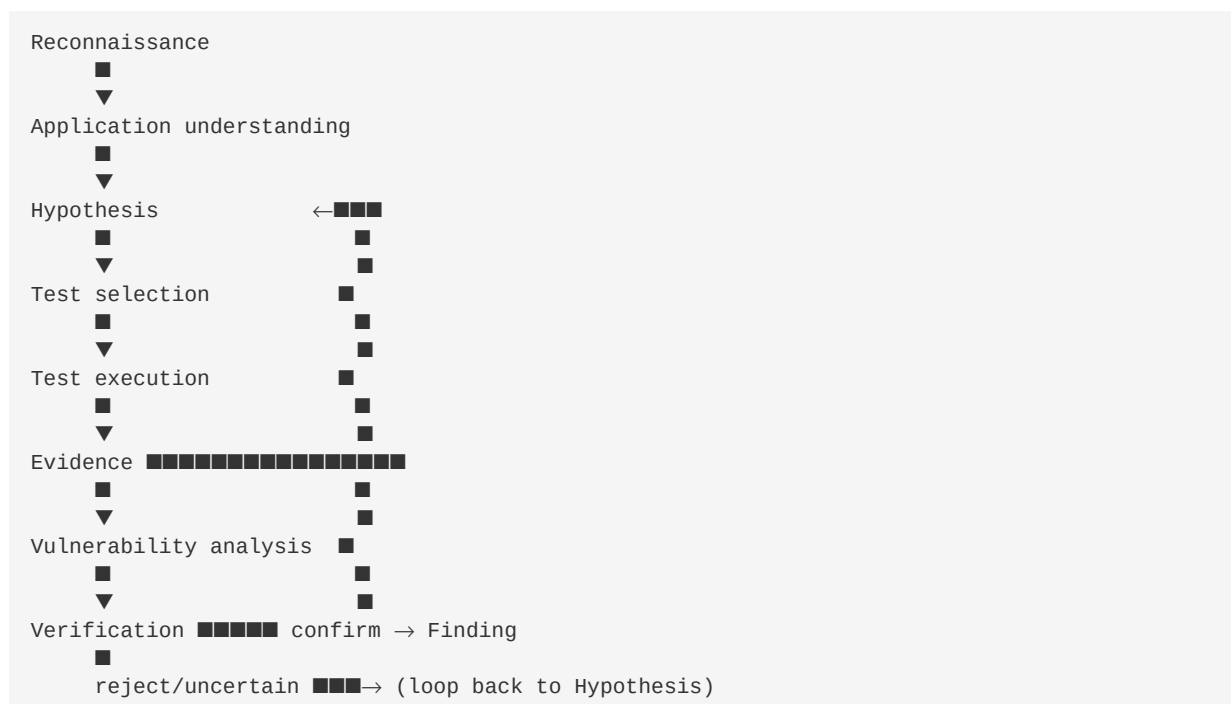
1.2 The AIPEN Concept

AIPEN (AI PENetration-testing) is designed as a **security-engineering system in which AI participates in a controlled testing pipeline**, not as an LLM wrapper around security tools.

The governing architectural principle is:

AI reasons about evidence; deterministic tools generate the evidence.

The system implements a closed **reasoning → action → observation → verification loop**:



A finding is only produced when it survives independent verification against collected evidence. **No evidence, no finding.**

1.3 Objectives

#	Objective
O1	Build a working AI security engineer that assists a human pentester in testing web applications , not a demo or proof-of-concept wrapper.
O2	Implement the full pentesting loop — observe, reason, plan, act, verify — as executable pipeline logic.
O3	Support vulnerability classes: SQL Injection, XSS, IDOR/BOLA, SSRF, and common API vulnerabilities in v1.
O4	Gate every AI-proposed action through a deterministic policy engine with per-target scope enforcement.
O5	Use a multi-provider AI layer (OpenAI, Claude, DeepSeek via AgentRouter; Gemini optional) with cost-aware routing, within a USD 50 total development budget.
O6	Operate on a Kali Linux environment, orchestrating industry-standard CLI security tools rather than reimplementing them.
O7	Provide a minimal web UI for target management, loop monitoring, human-in-the-loop approvals, and findings review.
O8	Evaluate the system against a known-vulnerable benchmark application (OWASP Juice Shop) and measure detection quality, false-positive rate, and cost efficiency.
O9	Architect for future commercialization as a bug-bounty-assistant product.

1.4 Non-Objectives (Out of Scope for v1)

- Network-layer penetration testing beyond basic service discovery (no exploitation of network services, no Active Directory).
- Mobile application testing.
- Source-code/static-analysis-based detection (SAST). AIPEN is a dynamic (DAST-style) system.
- Local/self-hosted LLM inference. All AI providers are remote APIs.
- Autonomous exploitation (post-exploitation, privilege escalation chains, data exfiltration).
- Multi-user support, authentication, or multi-tenancy.
- Container-based (Docker) sandboxing of tools.

1.5 Intended Use

Use case	Description
Bug bounty hunting (primary)	The owner runs AIPEN against in-scope bug-bounty targets to accelerate recon, hypothesis generation, and evidence-backed findings.
Benchmark/evaluation	The owner runs AIPEN against intentionally vulnerable applications (OWASP Juice Shop, and optionally DVWA/WebGoat) to measure and improve capability.
Commercialization research	The architecture and evaluation results inform a possible future commercial product (see §11).

1.6 Definitions

Term	Definition
Finding	A vulnerability that has been verified against collected evidence by an independent review step. The only output that reaches the report.
Hypothesis	An AI-generated, evidence-grounded suspicion that a specific endpoint/parameter may exhibit a vulnerability class. Never reported directly.
Evidence	Raw, reproducible artifacts: HTTP request/response pairs, response diffs, timing measurements, tool output — stored immutably with timestamps.
Verification	A distinct AI (or deterministic) step that challenges a hypothesis against evidence and votes confirm / reject / uncertain.
Scope	The explicitly configured set of domains, IPs, paths, ports, test types, and rate limits a target is allowed to receive.
Policy Engine	Deterministic allow/deny layer that every proposed action must pass before execution. The AI can never bypass it.
HITL	Human-in-the-loop: destructive or high-risk actions pause for human approval in the UI.
Loop iteration	One full pass of observe → reason → plan → act → verify for a target or endpoint.
AgentRouter	Third-party API gateway providing unified access to OpenAI, Claude, and DeepSeek models.
BOLA	Broken Object Level Authorization (API equivalent of IDOR).

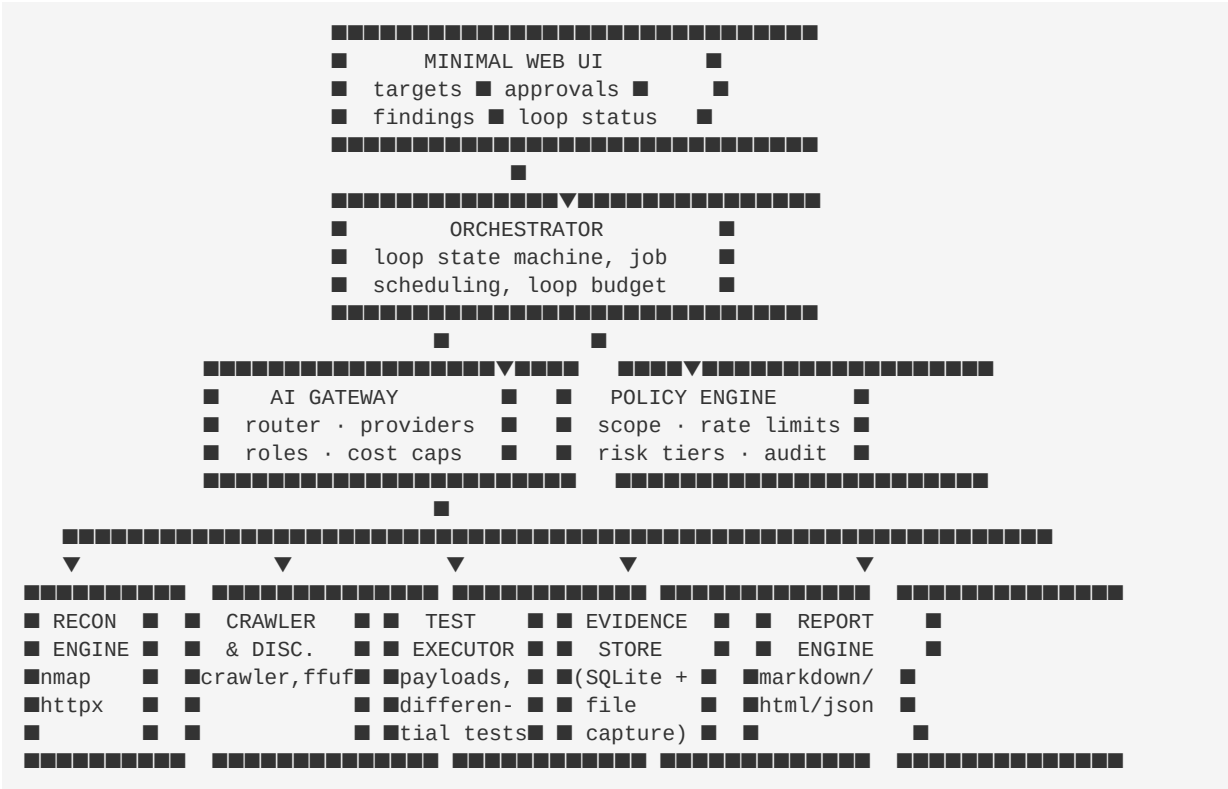
2. Project Overview

2.1 Product Summary

AIPEN is a single-operator, locally hosted web application that takes a target URL and an explicit scope configuration, then autonomously (under policy and human oversight) performs reconnaissance, crawls the application, generates vulnerability hypotheses, executes controlled tests, collects evidence, verifies results, and produces a professional findings report.

The operator interacts with the system through a minimal web dashboard: define target → review/approve high-risk actions → watch the loop work → review verified findings.

2.2 High-Level Architecture



2.3 Key Architectural Decisions

Decision	Rationale
Python 3.12 as the implementation language	Richest security-tooling ecosystem; all orchestrated tools have Python bindings or clean CLI contracts.
Orchestrate, don't reimplement	Nmap, httpx, ffuf, Nuclei, ZAP, sqlmap already solve detection sub-problems well. AIPEN adds the layer they lack: reasoning, planning, and verification.
SQLite in v1, PostgreSQL-ready schema	Solo operator, single machine, no infra. Schema uses portable SQL; migration path documented. No Redis (avoid extra services).
FastAPI + lightweight server-rendered UI	Fast async API layer; minimal UI via Jinja2 templates + HTMX (no heavy frontend build chain) keeps the stack small and maintainable by one person.

Decision	Rationale
AgentRouter-first AI access	One gateway, three providers (OpenAI, Claude, DeepSeek). Provider-agnostic interface keeps the door open for Gemini and future providers without rewrites.
Cost-aware model routing	USD 50 total budget forces discipline: cheap models for volume tasks, expensive models only for hard reasoning and verification.
Policy engine before AI integration	Built first, tested first. Safety is a foundation, not a feature.
No Docker	Runs directly on Kali Linux; tools invoked as CLI subprocesses with timeouts. Sandboxing deferred (see §11).

3. Planning Phase

3.1 Development Model

Agile (iterative), 2-week sprints, 6 sprints over 3 months.

Rationale: the pentesting loop has natural seams (recon → evidence → testing → reasoning → reporting) that map to sprint deliverables, and the owner needs working software early to start dogfooding against Juice Shop. Requirements are stable enough that pure Scrum ceremony (daily standups, etc.) is replaced by a lightweight personal kanban per sprint: backlog → in-progress → done, with a sprint review and retrospective done solo at each sprint boundary.

3.2 Sprint Plan

Sprint	Weeks	Theme	Deliverables (Definition of Done)
S1	1–2	Foundation & Safety	Repo scaffold; DB schema migrated; policy engine with scope config + rate limits, unit-tested; AI gateway with AgentRouter provider, health check, cost tracker; orchestrator skeleton that can run a no-op pipeline end-to-end.
S2	3–4	Recon & Evidence	Nmap + httpx recon modules; crawler + ffuf discovery module; raw evidence capture (request/response storage); recon summarisation prompt pipeline; evidence queryable from DB.
S3	5–6	Test Execution Engine	HTTP test executor with differential testing (baseline vs. mutated request); SQLi test module (error-based + time-based probes); XSS test module (reflected/stored detection); sqlmap integration as deep-verification tool for SQLi candidates.
S4	7–8	The Loop + AI Analyst	AI Planner role; AI Analyst role (hypothesis generation from evidence); AI Verifier role with confirm/reject/uncertain votes; loop closure: hypothesis → test → evidence → verification → finding, fully automatic for low-risk actions; Nuclei integration for template-based signals feeding the Analyst.

Sprint	Weeks	Theme	Deliverables (Definition of Done)
S5	9–10	API Vulns, HITL & UI	IDOR/BOLA and SSRF test modules; API endpoint handling (REST, JSON bodies, common auth patterns); risk-tier classification and HITL approval flow in UI; minimal dashboard: target create/start, loop status, approvals queue, findings list.
S6	11–12	Hardening, Reports & Evaluation	Report engine (Markdown + HTML); full Juice Shop benchmark run; evaluation metrics collected; false-positive tuning; documentation; v1.0 release tag.

3.3 Milestones

Milestone	Date (relative)	Exit criteria
M1 — Safe skeleton	End of S1	A scan job can be created and immediately blocked by policy when out of scope; AI gateway returns a routed response with cost logged.
M2 — Eyes and memory	End of S2	Recon on Juice Shop yields endpoints, parameters, tech stack; all raw evidence stored and retrievable.
M3 — Hands	End of S3	Differential test executor runs SQLi/XSS probes; sqlmap invoked on candidates; evidence diffs recorded.
M4 — Brain	End of S4	A full unattended loop pass on Juice Shop produces at least one verified finding with zero false positives in the findings list.
M5 — Product	End of S5	Human can drive the whole workflow from the UI; high-risk actions pause for approval.
M6 — Evidence of value	End of S6	Benchmark results documented (§8); report generated; v1.0 tagged.

3.4 Effort Estimates (solo, part-time-realistic)

Area	Share of effort
Test execution engine + vulnerability modules	~30%
AI roles (prompts, structured output, verification)	~20%
Orchestrator + policy engine	~15%
Recon/discovery integrations	~10%
UI + HITL	~10%
Evaluation + tuning + docs	~15%

3.5 Budget Plan (USD 50, development period)

Item	Allocation	Controls
AI API spend (AgentRouter: OpenAI/Claude/DeepSeek)	\$40	Per-call cost logging; per-scan budget cap (default \$0.50/scan, configurable); model routing (DeepSeek default for volume tasks); prompt caching where supported; verification only on promising hypotheses.
Domain/infra (optional Juice Shop VPS or bug-bounty adjacent costs)	\$10	Only if needed; Juice Shop runs locally for free.

Hard stops: if projected spend exceeds 70% of the remaining budget, the system switches all non-verification roles to the cheapest available model and prompts the operator.

3.6 Tools Selection (2 industry-standard tools per attack stage)

AIPEN orchestrates tools as CLI subprocesses on Kali Linux. Each stage uses exactly two primary tools, chosen for coverage overlap and CLI cleanliness.

Attack stage	Tool 1	Tool 2	Role in AIPEN
Service & host recon	Nmap	httpx (ProjectDiscovery)	Nmap: open ports, service versions. httpx: HTTP probing, status, titles, tech fingerprints, TLS metadata.
Content & endpoint discovery	ffuf	Katana (ProjectDiscovery crawler)	ffuf: content/directory brute-force on in-scope paths. Katana: JS-aware crawling to enumerate endpoints, parameters, forms.
Vulnerability signal scanning	Nuclei	OWASP ZAP (baseline/AScan via API)	Nuclei: fast template-based signals (fed to AI Analyst as <i>signals</i> , never direct findings). ZAP: passive + targeted active scan on discovered endpoints.

Attack stage	Tool 1	Tool 2	Role in AIPEN
Deep SQLi verification	sqlmap	custom differential prober	sqlmap: confirms/exploits SQLi candidates escalated by the loop; custom prober: cheap first-pass error/time-based checks that generate evidence.

Tool contract: every tool integration is a thin adapter implementing a common interface (`run(target_ctx)` → structured results + raw evidence files), with timeouts, output parsing, and failure capture. If a tool is missing on the host, the adapter degrades gracefully and logs the gap.

3.7 Environments

Environment	Purpose	Notes
Dev	Daily development	Kali Linux VM; Juice Shop via Docker on the target side (AIPEN itself does not use Docker) or <code>npm start</code> .
Benchmark	Evaluation runs (§8)	Fresh Juice Shop instance per run; reset between runs for reproducibility.
Production (personal)	Real bug-bounty targets	Strictest scope configs; HITL on for all medium+ risk actions; full audit logging.

4. Requirements Specification

4.1 Functional Requirements

Format: FR-<area>-<n> — MoSCoW priority (M = must, S = should, C = could).

A. Target & Scope Management

ID	Requirement	Pri
FR-SCOPE-1	The operator can create a Target consisting of: base URL(s), allowed domains, allowed IPs, allowed ports, allowed path prefixes, excluded paths, rate limit (req/sec), allowed test types, forbidden actions, and per-scan AI budget cap.	M
FR-SCOPE-2	Every outbound action (HTTP request, tool invocation) is validated against the active scope before execution; violations are blocked and logged.	M
FR-SCOPE-3	Scope supports wildcards for subdomains (e.g., *.example.com) and explicit deny-lists (logout, password-reset, payment endpoints default-denied).	M
FR-SCOPE-4	A target can be archived; archived targets refuse new scans but retain history.	S

B. Reconnaissance

ID	Requirement	Pri
FR-RECON-1	Execute Nmap service discovery against in-scope hosts; parse and store open ports/services.	M
FR-RECON-2	Execute httpx probing; store status codes, page titles, server/Tech fingerprints.	M
FR-RECON-3	Crawl the target (Katana) to enumerate endpoints, HTTP methods, parameters, and forms; store in the endpoint inventory.	M
FR-RECON-4	Run ffuf content discovery seeded from recon results; store discovered paths.	M
FR-RECON-5	Summarise recon output into an Application Profile (tech stack, entry points, auth surfaces, upload surfaces, API indicators) via the AI layer.	M

ID	Requirement	Pri
FR-RECON-6	Passively fingerprint JS libraries and API routes from crawled pages (feeds API vulnerability testing).	S

C. The Pentesting Loop

ID	Requirement	Pri
FR-LOOP-1	The orchestrator runs the loop: observe (query evidence store) → reason (AI Analyst generates hypotheses from evidence) → plan (AI Planner ranks next tests) → act (policy check → execute test → capture evidence) → verify (AI Verifier votes on hypotheses).	M
FR-LOOP-2	Each target run has a configurable loop budget : max iterations, max wall-clock time, max AI spend. Exhaustion stops the loop cleanly with partial findings preserved.	M
FR-LOOP-3	The Planner receives: application profile, endpoint inventory, existing hypotheses/findings, coverage map, and remaining budget; it outputs an ordered test plan with per-test justification.	M
FR-LOOP-4	Uncertain verification votes trigger a follow-up differential test chosen by the Planner (disagreement drives investigation, not voting).	M
FR-LOOP-5	The loop records every decision (planner output, test choice, verifier vote) with the exact model, prompt version, and cost for post-hoc audit.	M
FR-LOOP-6	The operator can pause/resume/abort a running loop from the UI.	S

D. Vulnerability Test Modules (v1 classes)

ID	Requirement	Pri
FR-TEST-1	SQLi module : error-based and time-based probes on all discovered parameters; differential comparison vs. baseline; escalation of strong candidates to sqlmap for confirmation.	M

ID	Requirement	Pri
FR-TEST-2	XSS module: reflected and stored XSS probes with canary payloads and context-aware encoding checks; evidence = reflection context + payload execution marker.	M
FR-TEST-3	IDOR/BOLA module: object-reference manipulation (sequential/GUID variants, cross-account where two sessions exist) on parameters named or typed as identifiers; evidence = cross-boundary data access in response.	M
FR-TEST-4	SSRF module: out-of-band interaction probes (canary URLs) in URL parameters and common SSRF-capable fields; evidence = callback received.	M
FR-TEST-5	API vulnerability module: JSON-aware testing of REST endpoints discovered by the crawler — BOLA, mass assignment, verbose error leakage, unauthenticated access to protected routes, rate-limit absence noted.	M
FR-TEST-6	Nuclei and ZAP results are ingested as signals that seed hypotheses for the AI Analyst — never promoted directly to findings.	M
FR-TEST-7	Every executed test stores: full request, full response, diff vs. baseline, timing, and tool raw output where applicable.	M

E. AI Layer

ID	Requirement	Pri
FR-AI-1	A provider-agnostic AI gateway exposes role-based generation: <code>generate(role, prompt, schema) → structured output</code> , where roles include planner, analyst, attacker, verifier, reporter, classifier.	M
FR-AI-2	Provider integrations: OpenAI, Claude, DeepSeek via AgentRouter; Gemini pluggable without touching call sites.	M

ID	Requirement	Pri
FR-AI-3	Model routing: each role maps to a model tier (volume → cheapest reliable; verification → different provider/model than the hypothesis author; hard reasoning → premium). Operator-configurable.	M
FR-AI-4	All AI outputs for analysis/verification roles must be structured (JSON-schema validated); invalid output triggers one retry then a failure record.	M
FR-AI-5	The Verifier must be able to reject the Analyst's hypothesis; rejections require a stated reason referencing evidence IDs.	M
FR-AI-6	Cost tracker logs per-call tokens, model, USD cost; enforces per-scan and global budget caps.	M
FR-AI-7	Disagreement handling: conflicting model assessments produce UNCERTAIN and spawn a Planner-selected follow-up test (\$5.5).	M
FR-AI-8	Prompts are versioned (prompt registry) so findings can be traced to the exact prompt that produced them.	S

F. Policy Engine & Human-in-the-Loop

ID	Requirement	Pri
FR-POL-1	Risk tiers: LOW (read-only probes, standard param fuzzing) auto-approved; MEDIUM (auth-boundary tests, cross-account IDOR probes, OOB callbacks) require approval if HITL enabled; HIGH (state-changing requests, admin-path access, any destructive verb) always require approval.	M
FR-POL-2	Approval requests surface in the UI with: action, endpoint, payload, reason from Planner, expected effect, risk tier. One-click approve/reject.	M
FR-POL-3	Rate limits are enforced by the executor regardless of AI behaviour (token bucket per host).	M
FR-POL-4	Every allow/deny/approval decision is written to an immutable audit log.	M

ID	Requirement	Pri
FR-POL-5	Forbidden action list is checked before execution even for approved items (operator overrides require editing scope, never a per-call bypass flag).	M

G. Evidence & Findings

ID	Requirement	Pri
FR-EV-1	Evidence records are immutable once written (append-only); corrections are new records superseding old ones.	M
FR-EV-2	Findings are created only from a confirmed verification vote linked to ≥ 1 evidence records.	M
FR-EV-3	Each finding carries: title, severity (CVSS-style qualitative + numeric where derivable), affected endpoint, description, evidence references, reproduction steps, impact, remediation guidance, confidence, and the full verification audit trail.	M
FR-EV-4	The findings list distinguishes: hypotheses (dormant), tests (executed), evidence (collected), findings (verified) — counts visible in UI.	M
FR-EV-5	Rejected hypotheses are retained with the rejection rationale (enables FP-rate measurement, §8).	M

H. Reporting

ID	Requirement	Pri
FR-REP-1	Generate a per-target report in Markdown and HTML : executive summary, findings ordered by severity, full evidence appendix.	M
FR-REP-2	Report exports include reproduction steps suitable for a bug-bounty submission (cURL commands, request/response pairs).	M
FR-REP-3	JSON export of findings for tooling integration.	S

I. UI

ID	Requirement	Pri
FR-UI-1	Single-page dashboard: targets list, create-target form, run controls (start/pause/abort), loop status (current phase, iteration, spend), approvals queue, findings table, findings detail view.	M
FR-UI-2	Live loop log tail (decisions, tests, votes) without page reload.	S
FR-UI-3	Read-only report preview in-app.	S

4.2 Non-Functional Requirements

ID	Requirement	Target
NFR-1	Safety	100% of executed actions pass policy; zero out-of-scope requests in audit review.
NFR-2	Reproducibility	Any finding can be re-derived from stored evidence + recorded prompts without rerunning the loop.
NFR-3	Cost	Full Juice Shop benchmark run ≤ \$2.00 AI spend; typical bug-bounty recon+first-pass loop ≤ \$0.50.
NFR-4	Performance	Recon phase ≤ 15 min for a typical single-host web app; loop iteration ≤ 3 min average.
NFR-5	Reliability	Tool adapter failures degrade gracefully (log + skip + notify); loop never crashes unrecoverably — state persists, resume possible.
NFR-6	Observability	Structured logs for every component; every AI decision traceable to model+prompt+evidence IDs.
NFR-7	Portability	Runs on a fresh Kali install with only documented apt/tool installs; no Docker for AIPEN itself.
NFR-8	Maintainability	One-person codebase; module boundaries per §5; typed Python (mypy-checked) with ruff.
NFR-9	Data durability	SQLite with WAL mode; automatic backup of DB file before each run.
NFR-10	Latency of AI calls	Volume-role calls ≤ 30 s p95; verification calls ≤ 90 s p95.

5. System Design

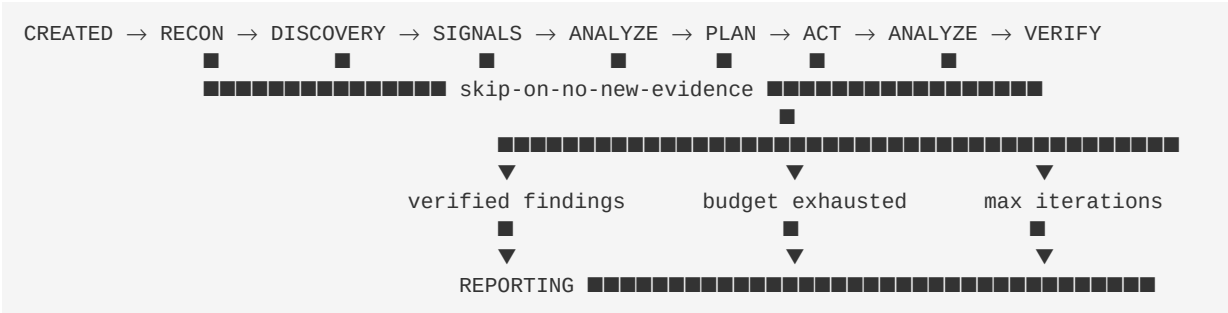
5.1 Component Breakdown

Component	Responsibility	Key interfaces			
Orchestrator	Loop state machine; job scheduling (async tasks); budget enforcement; phase transitions; pause/resume/abort.	start_run(target_id), pause_run(), event bus			
Recon Engine	Wraps nmap, httpx; produces Host/Service records.	run_recon(target) → ReconResult			
Discovery Engine	Wraps Katana (crawl) and ffuf (fuzz); produces Endpoint records.	run_discovery(target, recon) → Endpoint[]			
Signal Ingestion	Wraps Nuclei and ZAP; produces Signal records (never findings).	run_signals(target, endpoints) → Signal[]			
Test Executor	HTTP client with token-bucket rate limiter; payload injection; differential comparison (baseline vs mutated); sqlmap escalation.	execute(test_plan) → Evidence[]			
AI Gateway	Provider adapters (AgentRouter→OpenAI/Claude/DeepSeek, Gemini slot), role registry, structured-output enforcement, retries, cost tracking.	generate(role, prompt, schema, budget_ctx) → StructuredResult			
AI Roles	Prompt + output-schema definitions for planner, analyst, attacker, verifier, reporter, classifier.	Declarative role configs in prompts/			
Policy Engine	Pure function: (action, scope, risk_tier, approval_state) → ALLOW	DENY	NEEDS_APPROVAL `. No network, no AI — fully unit-testable.	check(action, ctx) → Decision	
Evidence Store	SQLite persistence + raw artifact files; append-only evidence; query API for the loop and UI.	save(evidence), query(filters)			

Component	Responsibility	Key interfaces
Report Engine	Jinja2 templates → Markdown + HTML (+JSON export).	render(target_id, fmt)
API Layer	FastAPI REST + server-rendered UI (Jinja2 + HTMX); WebSocket/SSE for live loop log.	HTTP
Tool Adapters	One thin module per CLI tool: builds command, runs subprocess with timeout, parses output to structured results + raw files.	ToolAdapter protocol

5.2 The Loop State Machine

A Run advances through phases; within the ANALYZE/PLAN phases the AI roles operate; every transition is logged.



State persistence: the run's full state (phase, iteration, pending approvals, budget consumed) lives in the DB, so abort/restart of the AIPEN process is recoverable.

Iteration semantics (one loop cycle):

- 1. **OBSERVE** — Query evidence store for evidence not yet analysed for this run.
- 2. **REASON (Analyst)** — Generate hypotheses: {endpoint, vuln_class, rationale, confidence, suggested_tests[], evidence_ids[]}. Constrained to v1 vuln classes; rationale must cite evidence IDs.
- 3. **PLAN (Planner)** — Rank pending hypotheses × suggested tests against coverage and budget; emit an ordered plan of concrete TestActions.
- 4. **ACT** — For each TestAction: Policy Engine check → (approval if needed) → Executor runs baseline + mutated request → Evidence saved. Rate limiter applied at the socket level.
- 5. **VERIFY (Verifier)** — An independent model reviews each hypothesis against its linked evidence and votes CONFIRM | REJECT | UNCERTAIN with a reason citing evidence IDs. UNCERTAIN → Planner proposes a follow-up differential test (max 2 follow-ups per hypothesis, then REJECT as unverifiable-in-budget).

5.3 Data Model

SQLite (WAL), append-only evidence. Schema is portable to PostgreSQL (no SQLite-only features in table defs).

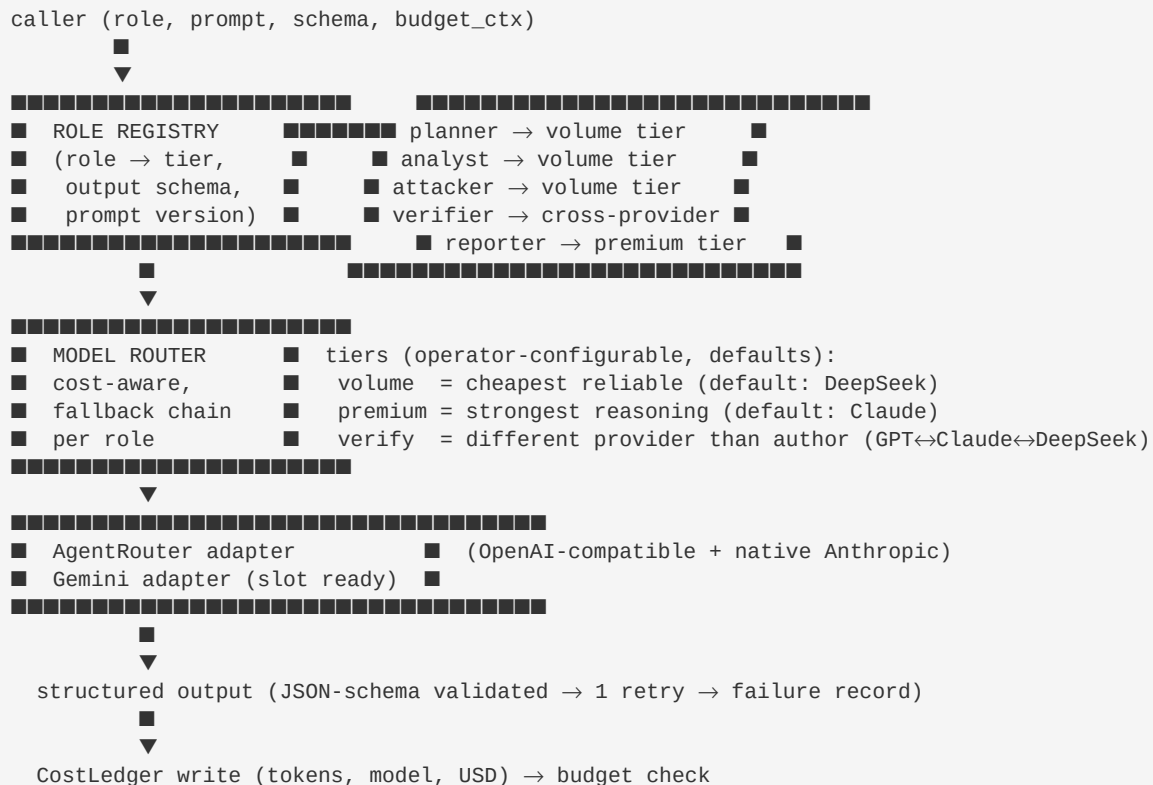
```
Target(id, name, base_urls, scope_config JSON, hitl_enabled, budget_cap_usd, created_at,
archived)
Run(id, target_id, state, iteration, budget_spent_usd, started_at, ended_at, config JSON)
Host(id, target_id, ip, hostname, source)
Service(id, host_id, port, protocol, name, version, banner)
Endpoint(id, target_id, url, method, params JSON, form_fields JSON, is_api, auth_required,
source, first_seen)
AppProfile(id, target_id, tech_stack JSON, entry_points JSON, auth_surfaces JSON, notes, model,
prompt_version)
Signal(id, run_id, endpoint_id, tool, template_id, name, severity_hint, raw_ref, created_at)
Hypothesis(id, run_id, endpoint_id, vuln_class, rationale, confidence, author_model,
prompt_version, status[open|verified|rejected|stale], created_at)
TestAction(id, hypothesis_id, type, payload JSON, risk_tier, policy_decision, approval_state,
executed_at)
Evidence(id, test_action_id, kind[request_response|diff|timing|tool_output|oob_callback],
baseline_ref, mutated_ref, analysis JSON, created_at)
Verification(id, hypothesis_id, evidence_ids, voter_model, vote, reason, created_at)
FollowUp(id, hypothesis_id, trigger_verification_id, planned_test JSON, result_action_id)
Finding(id, run_id, hypothesis_id, verification_id, title, severity, cvss_hint, endpoint_id,
description, repro JSON, impact, remediation, confidence, created_at)
ApprovalRequest(id, run_id, action JSON, risk_tier, reason, expected_effect, state[
pending|approved|rejected], decided_at)
AuditLog(id, ts, actor[system|operator|model], event, detail JSON)
PromptVersion(id, role, version, template, schema JSON, created_at)
CostLedger(id, ts, run_id, role, provider, model, input_tokens, output_tokens, usd)
```

Entity relationships: Target 1■n Run 1■n {Signal, Hypothesis, Finding}; Hypothesis 1■n TestAction 1■n Evidence; Hypothesis 1■n Verification; Finding requires exactly one CONFIRM verification.

Invariants enforced at the data layer:

- I1: No Finding row may exist without a linked Verification.vote = CONFIRM.
- I2: Evidence rows are never updated or deleted (append-only).
- I3: Every TestAction has a policy_decision recorded *before* any Evidence can reference it.
- I4: Verifier model ≠ hypothesis author model for the same chain (enforced in gateway routing config).

5.4 AI Gateway & Model Routing



Routing rules:

- R1: Default all volume roles to the cheapest provider that passes a weekly quality spot-check.
- R2: Verifier always uses a different provider than the hypothesis author (independence beats strength).
- R3: Escalation to premium tier only when: verifier UNCERTAIN on a high-confidence hypothesis, or Planner flags complexity, or operator forces.
- R4: On provider error/rate-limit: single retry with exponential backoff, then fall back to next provider in role's chain; record fallback in CostLedger.
- R5: Hard budget stop per run and globally (\$3.5).

5.5 Multi-Model Disagreement Handling

Deliberately **not** majority voting. Conflicting assessments (UNCERTAIN, or verifier rejects a high-confidence hypothesis) trigger targeted investigation:

1. Disagreement recorded with both positions and cited evidence.
2. Planner receives the disagreement and selects the cheapest discriminating test (a differential probe designed to separate the two interpretations).
3. New evidence re-enters verification — once with each of the disagreeing models.
4. Max 2 follow-up rounds; unresolved → hypothesis marked stale (neither finding nor rejection) and reported as "requires manual review" in the report appendix.

5.6 Policy Engine Design

Deterministic, pure, fully unit-tested — no AI, no I/O inside the decision core.

```
def check(action: Action, scope: Scope, approvals: ApprovalState) -> Decision:
  # 1. Scope: host/port/path/test-type membership + deny-list
  # 2. Forbidden actions (state-changing verbs, default-denied paths)
  # 3. Risk tier classification (static rules: payload class × endpoint class)
  # 4. HITL: HIGH always NEEDS_APPROVAL; MEDIUM if target.hitl_enabled
  # 5. Rate limit token availability
  # → ALLOW | DENY(reason) | NEEDS_APPROVAL(reason)
```

Default-denied paths (configurable): /logout, /password-reset*, /payment*, /delete*, /admin/* (HIGH tier — approvable but never automatic). Default rate limit: 10 req/s per host, burst 20. All parameters captured in the target's scope_config.

5.7 Test Executor & Differential Testing

- Baseline capture: every endpoint under test first gets a benign request stored as baseline evidence.
- Mutation: module-specific payloads; for v1 classes, probes are drawn from a curated library (not LLM-generated per-request — generation is permitted only as *suggestions* reviewed into the library, keeping spend and danger low).
- Differential comparison: status code, response length delta, content markers (DB error regexes, reflection of canary, timing deltas > threshold), header anomalies.
- OOB channel: SSRF probes use a canary domain/interaction server (configurable; e.g., a free OOB service or self-hosted callback listener).
- sqlmap escalation: invoked with --batch --level=2 --risk=1 scoped to a single flagged parameter, with output parsed into evidence; never given free-form targets.

5.8 UI Design (minimal)

Server-rendered Jinja2 + HTMX, dark theme, five views:

1. **Dashboard** — targets, run states, loop phase per run, spend gauge, findings count by severity.
2. **Target setup** — scope form (domains, paths, limits, budget cap, HITL toggle).
3. **Run monitor** — current phase, iteration, live decision log (SSE), loop controls.
4. **Approvals** — queue with action/payload/reason/expected-effect cards; approve/reject.
5. **Findings** — table → detail (evidence viewer with request/response pairs, verification trail) → report export.

5.9 Repository Layout

```
aipen/
■■■ app/
■   ■■■ api/           # FastAPI routes + UI views
■   ■■■ orchestrator/  # loop state machine
■   ■■■ policy/        # policy engine (pure)
■   ■■■ recon/         # nmap, httpx adapters
■   ■■■ discovery/     # katana, ffuf adapters
■   ■■■ signals/       # nuclei, zap adapters
■   ■■■ testing/       # executor, differential, modules/{sqli,xss,idor,ssrf,api}
■   ■■■ ai/            # gateway, router, providers, roles, cost
■   ■■■ evidence/      # store, models
■   ■■■ reports/       # templates + renderer
■   ■■■ core/          # config, db, logging, schemas
■■■ prompts/           # versioned prompt registry (YAML)
■■■ tests/
■   ■■■ unit/          # policy, executor, router, modules
■   ■■■ integration/   # adapters vs recorded tool output
■   ■■■ system/        # full-loop runs vs Juice Shop
■■■ data/              # sqlite db + evidence artifacts (gitignored)
■■■ docs/              # this document + runbooks
```

6. Implementation Plan

6.1 Technology Stack

Layer	Choice	Reason
Language	Python 3.12 (typed, ruff, mypy)	Security ecosystem, async support
API/UI	FastAPI + Jinja2 + HTMX + SSE	Minimal JS build chain; async fits I/O-bound testing
DB	SQLite (WAL) via SQLAlchemy	Zero-infra; portable schema for future Postgres
Jobs	asyncio task groups + DB-persisted run state	No broker needed at solo scale
HTTP client	httpx (async)	Baselines, mutations, proxy support for future Burp handoff
AI access	AgentRouter SDK / OpenAI-compatible client	One client for three providers
CLI orchestration	subprocess + asyncio	Kali tools are CLI-first
Tests	pytest + respx (HTTP mocking) + recorded tool fixtures	Deterministic CI-less test runs
Packaging	uv or pip-tools, pyproject.toml	Reproducible env on fresh Kali

6.2 Coding Standards

- All Policy Engine and Executor code is pure/testable; no hidden I/O.
- Every AI call site passes through the gateway — **no direct provider SDK usage elsewhere** (enforced by code review + import linting).
- Pydantic models for all inter-component messages; schemas versioned.
- Every tool adapter: timeout, output-size cap, structured parse result or explicit `ToolFailure`.
- Secrets (API keys) via environment / `.env` (gitignored); never in DB or logs.

6.3 Sprint Exit Criteria (per sprint)

A sprint is done when: all its "must" FRs are implemented, unit tests pass (`pytest -q green`), the sprint's milestone demo runs on Juice Shop or a mock, and the sprint retro notes are written into `docs/devLog.md`.

6.4 Definition of Done (per feature)

1. Code + type checks + lint clean.
2. Unit tests for decision logic; integration tests with recorded fixtures for adapters.
3. Audit log entries emitted.
4. Cost impact recorded (if AI-touching).
5. Documented in `docs/` module README.

6.5 Build Order Within Sprints

S1 establishes the cross-cutting skeleton (config, DB, logging, policy, gateway, cost ledger) — everything after S1 hangs off it. S2–S3 are pure deterministic engineering (no AI complexity). S4 wires the AI roles and closes the loop. S5 extends vulnerability coverage and adds the human layer. S6 is measurement and polish. This ordering front-loads risk: the riskiest parts (AI reliability, verification quality) get the most remaining time when they land in S4.

7. Testing Strategy

Testing has two meanings for AIPEN and both are covered: (A) testing **the platform's own software quality**, and (B) testing **the platform's security capability** against vulnerable targets (formalised in §8).

7.1 Unit Testing

Area	Approach	Key cases
Policy Engine	Pure function tests, exhaustive matrix	scope edge cases (wildcard subdomains, trailing dots, path prefix vs exact), deny-list precedence, risk-tier boundaries, approval-state transitions, rate-limit exhaustion
Executor differential	Mocked httpx (respX)	baseline vs mutated diff detection (status, length, markers, timing), encoding handling, redirect policy
Test modules	Fixture responses per vuln class	each module fires correct probes for param type; evidence shape correct
AI gateway/router	Stubbed providers	routing rules R1–R5, schema validation retry, fallback chain, budget stop
Cost ledger	Deterministic token pricing table	per-run cap, global cap, rounding
Evidence store	In-memory + temp files	append-only enforcement (I2), invariant I1 (no finding without confirm)

7.2 Integration Testing

- **Tool adapters** run against *recorded* tool output fixtures (checked into the repo) so tests don't require Kali tools; a separate `tools-live` marker runs real binaries when present.
- **Policy + Executor**: proposed action → denied action never reaches the HTTP layer (assert zero requests in respX).
- **Loop integration**: scripted stub-analyst drives the full state machine; verifies phase transitions, follow-up rounds (max 2), budget exhaustion path, pause/resume persistence.

7.3 System Testing (against Juice Shop)

Full runs of AIPEN against a locally hosted Juice Shop instance, reset between runs. Validates end-to-end behaviour: recon actually enumerates the API, the loop iterates, findings land, reports render. System tests double as the first evaluation pass (§8).

7.4 Testing of the Platform Itself (self-security)

Check	Method
UI/API input validation	Fuzz own forms (ironic but necessary); SQLi/XSS against own DB-rendered views
Secret handling	Grep CI-able checks: no keys in repo, no keys in logs/audit entries
Local-only binding	Verify server binds to localhost by default; no auth needed <i>because</i> no exposure

Check	Method
Dependency audit	<code>pip-audit</code> in the dev loop

7.5 Test Documentation

Each sprint maintains: test case list in `tests/TESTCASES.md`, fixture catalog, and known-failures list. The Juice Shop system-test runbook (`docs/runbook-juiceshop.md`) defines exact reset procedure so results are comparable across runs.

8. Evaluation Methodology

8.1 Benchmark Design

Primary benchmark: **OWASP Juice Shop** (v16+), locally hosted, factory-reset before each run. Rationale: it contains deliberately planted instances of all five v1 classes (SQLi incl. UNION/time-based, reflected+DOM XSS, BOLA/IDOR via the API, SSRF in the profile photo URL, plus auth and mass-assignment flaws), and it is the de-facto standard for tooling comparison — without overfitting the *architecture* to it (AIPEN modules are driven by the generic evidence/loop design, not Juice Shop specifics).

Anti-overfit safeguards: no Juice Shop strings, routes, or payloads anywhere in the codebase; all benchmark-specific config lives outside the repo; at least one secondary target (DVWA or WebGoat) is run once in S6 as a generalisation check.

8.2 Metrics (decided)

Metric	Definition	Target (v1)
Detection rate (recall)	Verified findings ÷ benchmark-planted vulns of v1 classes present in the run's scope	≥ 70% on Juice Shop scoped run
False-positive rate	Rejected-by-verifier hypotheses ÷ all hypotheses	≤ 15%
Precision (finding quality)	Verified findings ÷ (verified findings + manual-review rejections)	≥ 85%
Cost per verified finding	Total AI spend ÷ verified findings	≤ \$1.50
Time to first verified finding	Run start → first CONFIRM	≤ 30 min
Loop efficiency	Executed tests ÷ planned tests (plan abandonments)	≥ 80%
Verification agreement	Verifier CONFIRM ÷ Analyst-confident hypotheses	tracked for calibration; no target — a diagnostic
Safety record	Out-of-scope requests observed ÷ all requests	0 (hard requirement)
Human override rate	Operator-rejected approvals ÷ approval requests	tracked; target ≤ 20% (HITL noise indicator)

All metrics are computed from the evidence/audit tables automatically by an `aipen evaluate` command — no hand-tallying.

8.3 Baseline Comparisons

To prove the AI layer adds value (and to quantify it for commercialization), S6 includes a baseline comparison table on the same Juice Shop scope:

1. Nuclei alone (raw template output, no AI triage) — measures FP reduction from the evidence/verify loop.
2. ZAP baseline/AScan alone — measures detection-rate lift from hypothesis-driven targeted testing.
3. AIPEN with volume-tier routing vs premium-only routing — measures cost saving from model routing on identical runs.

8.4 Evaluation Protocol

1. Reset Juice Shop; record version + seed state.
2. Run AIPEN with the standard benchmark scope config and HITL off for LOW/MEDIUM tiers (operator approves HIGH items immediately to keep runs comparable).
3. Repeat 3 runs; report median with spread.
4. Produce the metrics table (§8.2), the baseline comparison (§8.3), and a qualitative failure analysis: every missed planted vuln and every false positive gets a one-paragraph root-cause note filed in docs/evaluation/.
5. Feed root causes into the backlog — evaluation output is sprint input, not an appendix.

8.5 Acceptance Criteria for v1.0

- All "must" FRs implemented and tested.
- Safety record = 0 out-of-scope requests across all runs.
- Detection $\geq 70\%$, FP $\leq 15\%$, cost per finding $\leq \$1.50$ on the Juice Shop benchmark.
- One full report generated from a benchmark run with ≥ 3 verified findings.
- Total AI spend across the entire 3-month development $\leq \$50$.

9. Ethics, Legal & Safety

Even as a personal tool, AIPEN is an offensive-capability system and is engineered accordingly:

1. **Authorization discipline.** AIPEN is used only against (a) intentionally vulnerable applications, or (b) targets where the operator has explicit authorization (bug-bounty program rules, written client consent). This is an operator commitment recorded here as a usage policy, and enforced operationally by the scope system: a target cannot be scanned without an explicit, operator-authored scope config.
2. **Rate limiting and non-destruction.** Default rate limits, default-denied state-changing paths, and HIGH-tier HITL gates minimise the chance of service impact. AIPEN performs **verification**, not exploitation: no exfiltration, no persistence, no post-exploitation.
3. **Auditability.** The append-only evidence and audit trail means every action AIPEN ever takes is reconstructable — a legal and ethical safeguard, and a differentiator for future commercial use.
4. **Safe storage.** Findings data (which may include sensitive target data) stays local; the DB file should be encrypted at rest by the operator (OS-level disk encryption) and excluded from backups that leave the machine.
5. **Bug-bounty compliance.** Report exports are designed to match the evidence standard bounty programs expect, reducing dispute risk.
6. **No malware/exploit delivery.** AIPEN never generates or delivers weaponised payloads (no reverse shells, no RCE chains). If an RCE-class issue is suspected, the system reports the *indication* and halts for human handling.

10. Risk Management

Risk	Likelihood	Impact	Mitigation
AI hallucinates findings despite evidence gate	Medium	High	Verifier independence rule (R2); no-evidence-no-finding invariant (I1); FP metric tracked per sprint, prompt fixes batched in
API spend blows the \$50 budget	Medium	Medium	Per-run caps, volume-tier defaulting, hard global stop at 70% (\$3.5), cost visible in UI
Tool output parsing breaks on version updates	High	Low	Adapters isolate parsing; recorded fixtures catch changes; graceful ToolFailure degradation
Loop thrashing (uncertain→follow-up loops)	Medium	Medium	Max 2 follow-ups then stale; loop budget caps; Planner cost-aware
False sense of security (missed vulns)	Medium	High	Evaluation vs known-planted benchmark; "requires manual review" appendix; explicit coverage map per run
Operator error: overly broad scope on a real target	Low	Critical	Deny-list defaults, HIGH-tier HITL always-on, scope review screen before first run, audit trail
Juice Shop overfitting	Medium	Medium	\$8.1 safeguards; secondary target generalisation run
Solo-dev time overrun on S4 (AI roles)	Medium	Medium	S4 is mid-timeline by design with S5/S6 buffer; stub-analyst integration tests exist from S3 so the loop mechanics are ready before real models
Third-party gateway (AgentRouter) outage/change	Low	Medium	Provider-agnostic gateway; per-role fallback chains; adapter slot for direct provider SDKs

11. Future Work & Commercialization

Deferred by design (v1-excluded items with a path forward):

1. **Multi-user & multi-tenancy** — per-user scopes, auth, billing. Natural product step; schema already isolates by Target.
2. **Dockerised tool sandboxing** — per-run containers for tool isolation and reproducibility.
3. **Local model tier** — for zero-marginal-cost volume roles and air-gapped use.
4. **Burp Suite handoff** — export interesting evidence sessions for manual deep-dives; import manual findings for verification.
5. **Vuln-class expansion** — auth/logic flaws, CSRF, file upload, GraphQL-specific testing, LLM-app testing (prompt injection) as a v2 differentiator.
6. **Scheduled monitoring** — recurring re-scans of in-scope assets with diff-based alerting (bug-bounty regression hunting).
7. **Commercial packaging** — the evaluation data (§8) plus the audit/safety architecture is the product story: *verified findings with evidence trails and enforced scope*, targeting solo bug bounty hunters and small security teams first.

12. Appendices

12.1 Glossary

Term	Meaning
DAST	Dynamic Application Security Testing — testing a running application from outside
OOB	Out-of-band — a side channel (callback server) that observes a target's behaviour indirectly
Canary payload	A unique, trackable marker string used to detect reflection or execution
Differential testing	Comparing responses to semantically equivalent-but-different inputs to infer behaviour
CVSS	Common Vulnerability Scoring System — severity scoring standard
Signal	A scanner's raw match, treated as investigation input, never as a finding

12.2 References

- OWASP Juice Shop — <https://owasp.org/www-project-juice-shop/>
- OWASP Top 10 (2021) — <https://owasp.org/Top10/>
- OWASP API Security Top 10 (2023) — <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>
- Nmap — <https://nmap.org/> · httpx — <https://github.com/projectdiscovery/httpx>
- Katana — <https://github.com/projectdiscovery/katana> · ffuf — <https://github.com/ffuf/ffuf>
- Nuclei — <https://github.com/projectdiscovery/nuclei> · OWASP ZAP — <https://www.zaproxy.org/>
- sqlmap — <https://sqlmap.org/>
- AgentRouter (third-party multi-provider AI gateway) — operator-configured
- CVSS v4.0 — <https://www.first.org/cvss/v4.0/specification-document>

12.3 Decision Log

#	Decision	Choice	Date
D1	Document purpose	Personal engineering blueprint (not academic)	2026-09-16
D2	MVP vuln classes	SQLi, XSS, IDOR/BOLA, SSRF, API vulns	2026-09-16
D3	Stack	Python 3.12 · FastAPI · SQLite · HTMX	2026-09-16
D4	Tooling policy	2 industry-standard CLI tools per attack stage on Kali	2026-09-16
D5	AI providers	AgentRouter (OpenAI/Claude/DeepSeek) ± Gemini; no local models	2026-09-16
D6	Budget	USD 50 with per-run caps and hard stops	2026-09-16
D7	Targets	Juice Shop benchmark + authorized real targets	2026-09-16

#	Decision	Choice	Date
D8	Process	Agile, 6 × 2-week sprints, solo	2026-09-16
D9	Deliverable formats	Markdown (source) + PDF (rendered)	2026-09-16

End of document — v1.0. Change requests go through edits to this file with a version bump and a new Decision Log entry.